



Rapport de Soutenance

Genesis

NOVA

Aurelien FOURÉ
Timoteo CEDOLIN
Lucas BOURGEOIS
Noé BRÉDIF

2025 - 2026

Table des matières

1	Introduction	5
1.1	Présentation du projet	5
1.2	Présentation de l'équipe	5
2	Pour la première soutenance	7
2.1	La partie graphique	7
2.1.1	L'importance de l'interface dans le projet . . .	7
2.1.2	Conception de l'interface avec Glade	7
2.1.3	Fonctionnalités principales de l'interface	9
2.2	Fonction solver	11
2.3	Suppression des couleurs	12
2.3.1	Transformation en niveaux de gris	12
2.3.2	Binarisation de l'image : méthode d'Otsu . . .	12
2.4	Prétraitement	13
2.4.1	Morphologie : ouverture et fermeture	13
2.4.2	Enlever le bruit	14
2.5	Détection des composantes connexes	15
2.6	Fonction logique XNOR	16
2.6.1	Fonctionnement réseau neuronal	16
2.6.2	Retour sur notre fonction XNOR	16
2.6.3	L'entraînement du réseau	18
3	En vue de la prochaine Soutenance	19

4	Pour la soutenance finale	20
4.1	Fin des détections des mots	20
4.1.1	Suppressions des dessins	20
4.1.2	Suppressions des consignes	21
4.1.3	Plusieurs mots sur une ligne	21
4.1.4	Suppressions des énumérations et des mots courts	21
4.1.5	Lettres collées	22
4.2	Outil d'analyse de grilles d'images	23
4.2.1	Architecture, fonctionnement et principe de base	23
4.2.2	Convention de nommage des fichiers	23
4.2.3	Fonction d'extraction de position	24
4.2.4	Fonction de comparaison pour le tri	24
4.2.5	Parcours récursif des dossiers	24
4.2.6	Construction et écriture de la grille	25
4.2.7	Quelques statistiques du programme	25
4.2.8	Les problèmes rencontrés lors de l'implémentation	25
4.2.9	Résumé du programme	26
4.3	Affichage de la résolution de la grille	28
4.3.1	Annotation de l'image résolue	28
4.3.2	Surlignage et anticrênelage	29
4.4	Rotation automatique de l'image	30
4.4.1	Le principe de détection de l'angle optimal . .	30
4.4.2	Optimisation de la recherche d'angle	31
4.4.3	Résolution de l'ambiguïté de signe	31
4.4.4	Intégration avec ImageMagick	32
4.4.5	Défis techniques et solutions apportées	32
4.5	Intégration complète de la chaîne de traitement	34
4.5.1	Le flux de traitement automatisé	34
4.5.2	Gestion des chemins et des dépendances	35
4.6	Système neuronal	36
4.6.1	Normalisation des Images d'Entrée	36
4.6.2	Architecture du Réseau Neuronale	36
4.6.3	Fonctionnement Général du Système	37

4.6.4	Mécanisme de Récompense et de Punition . . .	38
4.6.5	Sauvegarde et Stockage des Poids du Réseau Neuronal	40

1 Introduction

1.1 Présentation du projet

Le but de ce projet est de concevoir un OCR (*Optical Character Recognition*) capable d'extraire automatiquement les informations contenues dans une image représentant une grille de mots cachés. Le programme doit non seulement identifier la grille et les lettres qu'elle contient, mais aussi détecter la liste des mots à retrouver.

Une fois ces éléments extraits, le système devra être en mesure de rechercher chaque mot dans la grille et d'en déterminer les coordonnées de début et de fin, afin d'indiquer précisément leur position ou, le cas échéant, signaler les mots non trouvés.

L'application disposera d'une interface utilisateur intuitive, permettant à l'utilisateur de sélectionner une image depuis ses dossiers. Après traitement, les résultats s'afficheront directement dans l'interface, présentant les mots détectés, leur statut (trouvé ou non trouvé) ainsi que, pour ceux identifiés, leurs coordonnées exactes dans la grille.

1.2 Présentation de l'équipe

Ce projet est développé par l'entreprise Genesis, qui en assurera la conception et la réalisation. Le produit final portera le nom de NOVA – Neural Optical Vision Analyzer, un acronyme évoquant à la fois la vision artificielle et la puissance d'analyse neuronale mise en œuvre dans le traitement de l'image. Ce nom reflète l'ambition du projet : créer un outil performant, intelligent et autonome capable d'analyser visuellement des grilles de mots cachés avec une grande précision.

Le projet NOVA – Neural Optical Vision Analyzer est développé par une équipe de quatre membres : Aurélien, Timoteo, Noé et Lucas.

Chacun possède un rôle bien défini afin d'assurer une progression efficace et équilibrée du projet.

- Aurélien est en charge de la partie solver, c'est-à-dire de la conception de l'algorithme permettant de rechercher automatiquement les mots dans la grille à partir des données extraites.
- Timoteo s'occupe du développement du réseau neuronal, dont le but est d'améliorer la reconnaissance des lettres et des zones de texte dans l'image.
- Noé travaille sur la détection des éléments visuels : il met en place les méthodes permettant d'identifier la grille, les lettres, ainsi que la zone des mots à trouver.
- Lucas est responsable de la création de l'interface utilisateur, qui permettra de charger une image, d'afficher les résultats et de rendre l'application simple et agréable à utiliser.



Aurélien Fouré



Timoteo Cedolin



Noé Brédif



Lucas Bourgeois

Grâce à cette répartition claire des tâches, chaque membre peut se concentrer sur son domaine de compétence tout en collaborant avec les autres. Cette organisation favorise une synergie d'équipe efficace et permet d'avancer de manière cohérente vers un objectif commun : la réalisation d'un OCR complet et fonctionnel pour l'analyse automatique de grilles de mots cachés.

2 Pour la première soutenance

2.1 La partie graphique

2.1.1 L'importance de l'interface dans le projet

L'interface graphique joue un rôle central dans ce projet, car elle constitue le principal point de contact entre l'utilisateur et le programme. Sans une interface claire, intuitive et réactive, même l'application la plus performante sur le plan technique risque d'être perçue comme complexe ou inutilisable. Dans le cadre de notre projet, l'interface n'est pas simplement un outil visuel : elle est la passerelle qui permet de rendre accessibles et compréhensibles les différentes fonctionnalités de traitement d'image, comme l'importation, la rotation et la conversion.

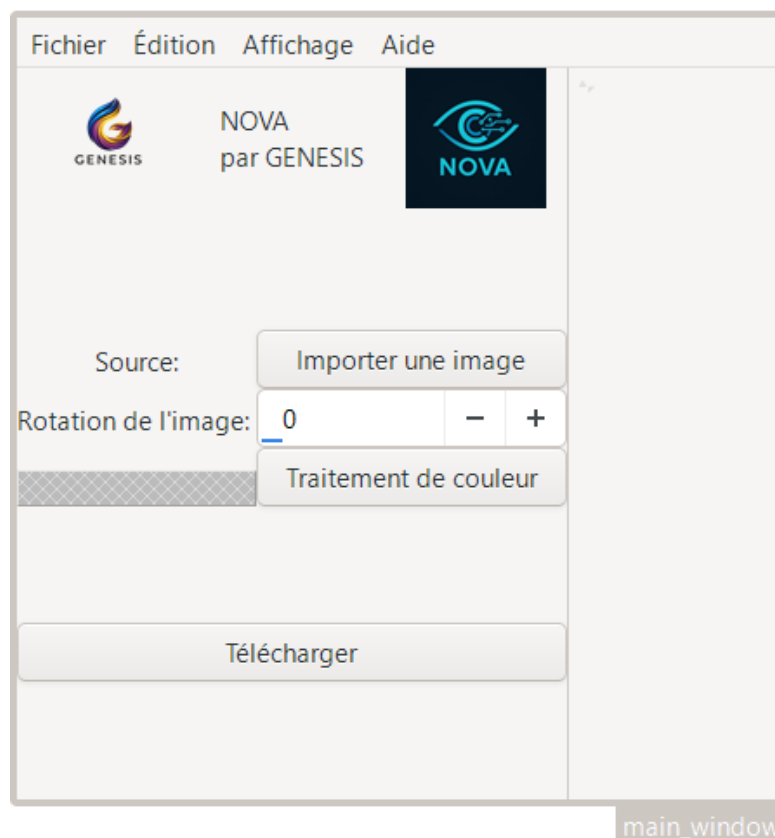
Elle a été pensée pour guider l'utilisateur tout au long du processus, depuis le chargement de l'image jusqu'à la sauvegarde du résultat. L'objectif principal est d'offrir une expérience fluide, où chaque action (cliquer sur un bouton, modifier un angle, lancer une conversion) se traduit immédiatement par un retour visuel clair. En d'autres termes, l'interface transforme des opérations techniques internes (gestion de fichiers, calculs de transformation, rendu graphique) en interactions simples et naturelles.

Grâce à cette approche, le programme devient accessible non seulement à des développeurs ou des étudiants en informatique, mais aussi à des utilisateurs non techniques souhaitant manipuler facilement des images sans connaître les détails de l'implémentation sous-jacente.

2.1.2 Conception de l'interface avec Glade

Pour concevoir l'interface graphique, nous avons utilisé Glade, un éditeur visuel dédié au développement d'interfaces GTK. Cet outil présente un avantage majeur : il permet de séparer clairement la logique du programme (le code en C) de la conception visuelle (l'organisation des éléments à l'écran).

Grâce à Glade, il est possible de créer la fenêtre principale, de positionner les boutons, les zones d'affichage d'image, les champs numériques et tous les autres composants sans écrire une seule ligne de code graphique. Cette séparation entre l'interface et le code permet un développement plus propre et plus modulaire. Le fichier généré par Glade, au format .glade (fichier XML), décrit toute la structure visuelle de l'application : les widgets, leur hiérarchie, leurs propriétés et leurs identifiants.



Dans le code C, nous chargeons ensuite cette interface à l'aide de la fonction `gtk_builder_add_from_file()`, ce qui nous permet de récupérer chaque élément (boutons, images, spinbuttons, etc.) par son identifiant et de lui associer des fonctions spécifiques via des signaux (`g_signal_connect()`).

L'utilisation de Glade apporte également un gain considérable en termes de maintenabilité et d'évolutivité. Si des modifications d'apparence sont nécessaires par exemple changer la disposition des boutons ou ajouter un nouveau champ, elles peuvent être réalisées directement dans Glade, sans modifier le code source. Cela rend la mise à jour de l'interface beaucoup plus rapide et réduit les risques d'erreurs logicielles. Enfin, cette approche facilite la collaboration entre les membres du projet : un développeur peut se concentrer sur la logique du programme tandis qu'un autre travaille sur l'apparence et l'ergonomie de l'application, tout en conservant une cohérence globale.

L'utilisation de Glade apporte également un gain considérable en termes de maintenabilité et d'évolutivité. Si des modifications d'apparence sont nécessaires—par exemple changer la disposition des boutons ou ajouter un nouveau champ—elles peuvent être réalisées directement dans Glade, sans modifier le code source. Cela rend la mise à jour de l'interface beaucoup plus rapide et réduit considérablement les risques d'erreurs logicielles. Enfin, cette approche modulaire a favorisé une véritable synergie d'équipe : chacun a pu travailler indépendamment sur son domaine de compétence tout en s'inscrivant dans une vision globale cohérente du projet.

2.1.3 Fonctionnalités principales de l'interface

L'interface que j'ai développée offre plusieurs fonctionnalités essentielles qui structurent le flux de travail de l'utilisateur. La première étape consiste à charger une image depuis le système de fichiers grâce au bouton d'importation. Un dialogue de sélection avec filtrage automatique des formats d'images supportés (PNG, JPEG, BMP, GIF) permet une sélection facile. Une fois l'image chargée, elle est immédiatement affichée à l'écran.

L'une des innovations majeures est la détection et correction automatique de l'inclinaison des images. Un clic sur le bouton « Rotation automatique » déclenche un algorithme sophistiqué qui analyse l'image, détecte son angle d'inclinaison et la redresse automatiquement. Cette fonctionnalité, détaillée dans les sections suivantes, répond à un besoin critique puisque les photos de grilles de mots cachés sont souvent prises avec un léger angle.

Le bouton de conversion en noir et blanc lance l'exécution du programme preprocess, qui traite l'image pour produire une version binaire optimisée pour la détection. Après conversion, l'image résultante est automatiquement rechargée et affichée, permettant à l'utilisateur de visualiser instantanément le résultat du traitement. Enfin, l'utilisateur peut exporter l'image traitée dans différents formats (BMP, PNG, JPEG) via le dialogue de sauvegarde.

Un soin particulier a été apporté à la gestion des formats d'images. GTK utilise la bibliothèque GdkPixbuf qui supporte nativement une grande variété de formats à l'importation. Pour la sauvegarde, j'ai implémenté une détection automatique du format basée sur l'extension du fichier choisi, rendant l'interface intuitive et évitant les erreurs courantes.

2.2 Fonction solver

Pour ma part, j'ai commencé par développer la fonction `solver`, dont le rôle est de lire une grille contenue dans un fichier texte ainsi que le mot à rechercher, puis de retourner les coordonnées de départ et de fin du mot dans la grille.

Cette première étape s'est révélée relativement simple, car elle repose sur un algorithme de recherche séquentielle dans un tableau généré à partir du fichier texte. Il s'agit essentiellement de parcourir la grille dans toutes les directions possibles (horizontale, verticale et diagonale) jusqu'à repérer la séquence de lettres correspondant au mot recherché. En quelques jours, cette partie du projet a pu être finalisée sans réelle difficulté technique.

```
int Check(int x,int y, int dx, int dy, char* word,int rows , int cols, char**grid)
{
    int x1 = x;
    int y1 = y;
    int i = 1;
    while(word[i] != '\0')
    {
        x += dx;
        y += dy;
        if(x < 0 || y < 0 || y >= rows || x >= cols)
            return 1;
        if(grid[y][x] != word[i])
            return 1;
        i++;
    }
    printf("(%d,%d)(%d,%d)\n",x1,y1,x,y);
    return 0;
}
```

```
aurel@pcsirow:/mnt/c/Users/aurel/Documents/Epita/OCR/Projet_Ocr/Solver$ cat grid
HORIZONTAL
DXRAHCLBGA
DIKCILEOKC
IGAJHYLYHI
HGFGODTIOT
GDLROWKBFR
PLNRDNERGE
JHAIDUJGV
UMGFFOLLEH
aurel@pcsirow:/mnt/c/Users/aurel/Documents/Epita/OCR/Projet_Ocr/Solver$ ./solver grid horizontal
(0,0)(9,0)
aurel@pcsirow:/mnt/c/Users/aurel/Documents/Epita/OCR/Projet_Ocr/Solver$ ./solver grid hello
(9,8)(5,8)
aurel@pcsirow:/mnt/c/Users/aurel/Documents/Epita/OCR/Projet_Ocr/Solver$ ./solver grid rien
Not found
aurel@pcsirow:/mnt/c/Users/aurel/Documents/Epita/OCR/Projet_Ocr/Solver$ |
```

2.3 Suppression des couleurs

2.3.1 Transformation en niveaux de gris

Pour transformer l'image en niveaux de gris, il faut parcourir tous les pixels.

Un pixel a plusieurs composantes dont le rouge, le vert et le bleu.

Il peut y en avoir d'autres, mais elles ne sont pas utilisées pour le projet.

Chaque pixels doit calculer une moyenne pondérée des trois premières composantes.

Puis on affecte le gris obtenu aux composantes de l'image.

2.3.2 Binarisation de l'image : méthode d'Otsu

La binarisation utilise un seuil calculé automatiquement selon la méthode d'Otsu.

Le but est transformer l'image avec les couleurs extremums :

Si le pixel est plus clair que le seuil alors il devient blanc, sinon il devient noir.

Les poids ω_i représentent la probabilité d'être dans la i -ème classe, chacune étant séparée par un seuil t . Finalement, les σ_i^2 sont les variances de ces classes.

$$\begin{aligned}\sigma_w^2(t) &= \omega_1(t)\sigma_1^2(t) + \omega_2(t)\sigma_2^2(t) \\ \sigma_b^2(t) &= \sigma^2 - \sigma_w^2(t) = \omega_1(t)\omega_2(t) [\mu_1(t) - \mu_2(t)]^2\end{aligned}$$

Otsu montre que minimiser la variance intra-classe revient à maximiser la variance inter-classe. Elle est exprimée en termes des probabilités de classe ω_i et des moyennes de classes μ_i qui à leur tour peuvent être mises à jour itérativement.

On calcule l'histogramme et les probabilités de chaque niveau d'intensité et on définit les $\omega_i(0)$ et $\mu_i(0)$ initiaux.

Pour chaque seuils possibles $t \in [[0; 256[[$, on met à jour ω_i et μ_i et on calcule $\sigma_b^2(t)$.

Le seuil désiré correspond au maximum des $\sigma_b^2(t)$.

2.4 Prétraitement

2.4.1 Morphologie : ouverture et fermeture

En Théorie, une figure est un amas de même valeur non nulle. En pratique, les écritures sont en noir sur fond blanc, un pixel sans couleur (en noir) a une valeur nulle, alors que un pixel coloré (plus claire) a une valeur non nulle.

Pour les opérations morphologiques suivantes, on garde la définition mathématique. Donc on considérera que les figures sont les zones claires. (le fond, les compteurs (trous des lettres), dessins clairs sur un objet foncé, mais pas les lettres !)

$X \subset \llbracket 0; w \rrbracket \times \llbracket 0; h \rrbracket = \{(0, 0), (0, 1), \dots, (w, h)\}$ l'ensemble des coordonnées d'une figure.

$B = \{-1, 0, 1\}^2 = \{(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1)\}$

Pour un pixel donné, B représentant les 4 cotés, les 4 diagonales et lui même.

La zone autour d'un pixel est donc au maximum de $|B| = 3 \times 3 = 4 + 4 + 1 = 9$ pixels.

Le translaté de B positionné sur un pixel p est $B_p = \{p + b \mid b \in B\}$.

Les exemples considèrent une main comme figure.

- Dilatation : élargir la figure.

Pour chaque pixel p , On prend la plus grande valeur (maximum) autour du pixel p et on l'affecte à p . (exemple : moufle)

$$\delta_B(X) = X \oplus B = \{p + b \mid b \in B, p \in X\} = \bigcup_{p \in X} B_p$$

- Érosion : rétrécir la figure.

Pour chaque pixel p , On prend la plus petite valeur (minimum) autour du pixel p et on l'affecte à p . (exemple : paume d'une petite main)

$$\epsilon_B(X) = X \ominus B = \{p \mid B_p \subset X\} = \bigcap_{b \in B} X_{-b}$$

- Ouverture : enlever les débordements.

La figure rétrécit puis s'agrandit à nouveau. On perd les pixels qui n'ont pas touché un pixel qui touche des 8 côtés la figure. (exemple : main sans doigts)

$$\gamma_B(X) = X \circ B = (X \ominus B) \oplus B = \delta_B(\epsilon_B(X))$$

- Fermeture : ajouter les encerclements.

La figure s'agrandit puis rétrécit à nouveau. On gagne les pixels qui touche des 8 côtés un pixel qui touche un pixel de la figure. (exemple : main palmée)

$$\phi_B(X) = X \bullet B = (X \oplus B) \ominus B = \epsilon_B(\delta_B(X))$$

- Médiane :

Pour chaque pixel p de l'image, On prend et trie les valeurs des pixels autour de p puis on récupère celui du milieu et on l'affecte à p .

(exemple : une main palmée avec des doigts très court)

- Tris puis choix :

On peut considérer un paramètre a pour connecter les 5 algorithmes précédents.

Pour l'érosion $a = 0$, pour la dilatation $a = 1$ et pour la médiane $a = 0.5$.

Il est alors possible de mettre ce paramètre à n'importe quel valeur dans $[0; 1]$.

On affecte à chaque pixel p la $\lfloor (\text{nombre_de_pixels_autour_de_}p - 1) \times a \rfloor$ -ième valeur de la liste triée des pixels autour de p (indexation basée sur 0).

On a recrée alors l'ouverture et la fermeture morphologique avec ce nouveau paramètre pour affiner le traitement.

2.4.2 Enlever le bruit

- Valeur moyenne :

Pour chaque pixel p de l'image, On somme les valeurs des pixels à une certaine distance de p , puis on divise par le nombre de pixels rencontrés et on affecte le résultat à p .

- Suppression brute des petites taches de pixels :

On supprime les taches de pixels de couleurs foncée lorsque le nombre de pixels de la tache est sous un certain seuil. On utilise un parcours en largeur avec une 8-connectivité pour que les diagonales soit comptées comme dans une même tache. Si la taille de la tache est au dessous du seuil, alors on dépile tous les pixels de cette tache et on les met en blanc.

2.5 Détection des composantes connexes

La détection automatique des éléments présents dans l'image.

L'objectif était d'identifier et d'extraire différentes zones :

la grille elle-même, les lettres qui la composent, la zone listant les mots à trouver, ainsi que les mots et leurs lettres respectives.

Cette partie s'est avérée bien plus délicate, car il existe de nombreuses approches possibles, traitement d'image, reconnaissance de formes, filtrage, etc.

mais notre contrainte principale était de n'utiliser que la librairie standard, ce qui a considérablement restreint les options disponibles.

Sans bibliothèques spécialisées comme OpenCV ou PIL, certaines méthodes classiques de traitement d'image devenaient beaucoup plus ardues à mettre en œuvre.

Nous avons donc pris le temps de réfléchir à la stratégie la plus adaptée dans ce contexte limité. Après plusieurs essais et discussions, nous avons opté pour une méthode basée sur la détection de densité pour distinguer les zones pertinentes de l'image.

L'idée était ensuite de repérer la plus grande composante connexe (avec une 8-connectivité), correspondant à la grille principale, afin de pouvoir isoler et traiter les autres éléments plus facilement. Cette approche, bien qu'un peu rudimentaire, nous a permis d'obtenir des résultats satisfaisants sans avoir recours à des outils externes.

On détecte les positions (colonne x ligne) de chaque case de la grille.

On utilise un double histogramme de positions centrales des composantes pour regrouper les lettres par ligne et colonne (avec une tolérance).

Ça permet aussi de séparer la grille du reste (comme des mots).

Enfin on assigne à chaque lettre un identifiant et sa position dans la grille

2.6 Fonction logique XNOR

2.6.1 Fonctionnement réseau neuronal

Un réseau neuronal fonctionne de la même manière qu'un vrai cerveau, c'est à dire qu'il a une couche d'entrée, une couche cachée et une couche de sortie.

La couche d'entrée est notre point de départ dans le système, dans notre exemple de la fonction XNOR, cela représente les entrées A et B ainsi que les valeurs qu'elles ont.

La couche cachée, quant à elle, permet de faire les calculs du code, c'est la partie ou le neurone "réfléchit" ce qui est demandé pour ensuite transmettre les données calculées à la couche de sortie.

Enfin, la couche de sortie donne la décision finale donc dans notre exemple elle permet de dire si l'opération logique donne le résultat binaire 0 ou 1.

Aussi, le réseau neuronal comporte des biais et des poids qui ont chacun leur importance. Les poids servent pour déterminer l'importance de chaque connexion entre les neurones. Les biais permettent d'améliorer l'ajustement du modèle. On peut faire une analogie avec l'étude d'un produit sur le marché avec comme critère les poids et comme condition minimal le biais. On peut faire un exemple avec l'achat d'une voiture où chaque poids n'est pas forcément le même (prix, couleur, marque) et le biais (condition minimal à respecter) pour la voiture en question.

2.6.2 Retour sur notre fonction XNOR

Dans notre fichier, on a donc plusieurs fonctions qui font chacun différentes choses. Pour commencer, on définit un struct avec ce dont on a besoin, une fonction qui initialise les valeurs de ce qu'on a dans le struct relié à un pointeur. Ensuite, une fonction qui fait les calculs de la fonction logique XNOR dans laquelle la couche cachée et la couche de sortie sont apparentes.

On a aussi une fonction qui est le point le plus important du réseau, autrement dit la fonction d'activation. C'est celle qui permet de faire les calculs pour avoir des valeurs aussi proche de 0 ou de 1 possible pour avoir une bonne représentation binaire. Cette fonction est donc appelée plusieurs fois au moment de faire les calculs dans la couche cachée et la couche de sortie. C'est la fonction Sigmoidale qui est représenté par $f(x) = \frac{1}{1+e^{-x}}$

```
Nombre de simulation d'entraînement (entre 1 et 100 000): 5000
```

```
=== Début de l'entraînement ===
```

```
simulation 0: Loss = 0.240518
```

```
simulation 1000: Loss = 0.179243
```

```
simulation 2000: Loss = 0.004658
```

```
simulation 3000: Loss = 0.001757
```

```
simulation 4000: Loss = 0.001062
```

```
simulation 4999: Loss = 0.000755
```

```
=== Entraînement terminé ===
```

```
Poids APRES l'entraînement:
```

```
=== Architecture du Réseau ===
```

```
Couche d'entrée: 2 neurones (A, B)
```

```
Couche cachée: 2 neurones
```

```
Neurone 1:
```

```
    w1=-6.1988, w2=6.2075, bias=-3.4338
```

```
Neurone 2:
```

```
    w1=-5.2708, w2=5.5258, bias=2.6114
```

```
Couche de sortie: 1 neurone
```

```
    w1=-8.5530, w2=8.3600, bias=-3.9579
```

```
Résultats APRES l'entraînement:
```

```
=== Table de Vérité !A.!B + A.B ===
```

A	B	!A.!B	A.B	Attendu	Résultat	Correct
0	0	1	0	1	1	✓
0	1	0	0	0	0	✓
1	0	0	0	0	0	✓
1	1	0	1	1	1	✓

```
Précision: 4/4 (100%)
```

2.6.3 L'entraînement du réseau

Afin d'entraîner le modèle à calculer la fonction XNOR, on dispose de trois éléments importants qui sont la fonction d'entraînement, la fonction qui permet l'apprentissage et la fonction qui calcule les erreurs et retourne un résultat pénalisant de sorte à faire moins d'erreur.

La fonction du calcul d'erreur est faite de sorte à ce que les erreurs aient de hautes valeurs pour que les grosses erreurs soient très pénalisées (erreur quadratique). Notre fonction d'entraînement est simplement une boucle for dans une autre où l'on applique la rétropropagation, calcule les pertes totales et enfin affiche ces pertes qui convergent vers 0.

Enfin, la rétropropagation qui est l'algorithme d'apprentissage se compose de six étapes. Premièrement, on calcule toutes les valeurs de sortie donc les différents tests de la fonction à faire. Ensuite, il calcule l'erreur de sortie pour savoir si le modèle a ou non fait des erreurs. Il ajuste ensuite le signal d'erreur pour savoir vers quel signe ajuster les poids. Il faut ensuite propager l'erreur aux neurones cachés. Enfin, il met à jour les poids en faisant le produit de l'ancien poids, du signal d'erreur et de la valeur prise par A et B dans notre fonction XNOR.

Ces éléments sont donc les plus importants pour entraîner notre modèle.

3 En vue de la prochaine Soutenance

Pour la prochaine soutenance, plusieurs objectifs majeurs ont été fixés afin d’aboutir à une version complète et pleinement fonctionnelle du projet NOVA – Neural Optical Vision Analyzer. Ces étapes représentent la finalisation de l’ensemble des modules principaux du système, depuis le traitement de l’image jusqu’à l’affichage des résultats dans l’interface.

Tout d’abord, nous devons mettre en place le prétraitement complet des images, comprenant notamment la rotation automatique. Cette étape est essentielle pour corriger les éventuelles inclinaisons de la grille dans l’image source et garantir une lecture optimale par le réseau neuronal. Ce module assurera ainsi une standardisation des images avant leur analyse.

Ensuite, le réseau de neurones devra être finalisé et rendu totalement fonctionnel. Il s’agira d’achever la phase d’apprentissage du modèle afin qu’il puisse reconnaître avec précision les lettres de la grille ainsi que celles de la liste des mots à trouver. Cette étape, véritable cœur intelligent du projet, permettra à l’application d’interpréter automatiquement le contenu visuel sans intervention humaine.

Une fois la reconnaissance effectuée, nous devrons passer à la reconstruction complète de la grille et de la liste de mots à partir des résultats du réseau neuronal. L’objectif est de reconstituer fidèlement les informations extraites afin de pouvoir les exploiter dans la phase suivante.

Vient ensuite la résolution de la grille, qui consiste à appliquer le module “solver” pour rechercher la position de chaque mot dans la grille reconstruite. Cette étape combinera les résultats du réseau neuronal et les algorithmes de recherche afin d’identifier les mots trouvés et leurs coordonnées exactes.

Une fois la résolution terminée, il sera nécessaire de développer l’affichage de la grille dans l’application, permettant à l’utilisateur de visualiser clairement les mots identifiés, leurs positions et le résultat global de l’analyse.

4 Pour la soutenance finale

4.1 Fin des détections des mots

Suite à la détection de la grille, des lignes de mots et la suppression des composantes trop larges ou grandes (ou trop petites), la détection des lettres des mots à trouver et leur regroupement fonctionnaient seulement pour des cas simples (les premiers niveaux des tests).

Cependant, il reste en plus des mots, les dessins, la consigne et les énumérations devant les mots qui brouillait l'algorithme sur le dernier niveau.

Pour les différencier, nous avons plusieurs algorithmes.

4.1.1 Suppressions des dessins

Les mots sont regroupés sur l'image en dehors de la grille. les lettres des mots sont donc toutes sur un des 4 cotés de la grille (en haut, en bas, à gauche ou à droite de la grille).

On calcule le barycentre (le centre de gravité ou la position moyenne) des points de toutes les composantes connexes restantes.

On utilise la boîte de l'ensemble de la grille : la plus petite boîte possible qui contient toutes les lettres de la grille (que l'on a précédemment créés).

On garde en mémoire les positions des 4 points cardinaux de l'ensemble de la grille.

On récupère le point cardinal de la grille qui est le plus proche du barycentre.

Enfin, on supprime les composantes qui sont du mauvais côté.

Par exemple, si une image a un dessin et un titre en haut, une image à droite, une grille au centre et les mots en dessous. Les lettres des mots ont un plus grand poids. Le barycentre se trouve donc vers le bas (et à l'intérieur) de la grille, légèrement à droite. Ce point moyen est donc plus proche du bas que des 3 autres points cardinaux de la grille. On peut alors supprimer toutes les composantes qui sont au dessus de la limite basse de la grille. Ici, il nous restera que les lettres des mots.

On fait de même pour les autres directions.

4.1.2 Suppressions des consignes

Il est possible que la consigne soit du même côté que les mots à trouver. Les Mots et phrases des consignes doivent être différenciés. Les Mots à trouver sont tous dans une même police de même hauteur. On calcule la médiane des hauteurs des lettres restantes. Puis on filtre ces lettres (avec une tolérance en y) pour garder seulement celles qui ont à peu près la même hauteur.

4.1.3 Plusieurs mots sur une ligne

Le regroupement des lettres par mots se fait dans la même idée que le regroupement des lignes de la grille, sauf qu'il peut avoir des longueurs de mots différentes. De plus, si il y a un écart important d'espacement entre deux lettres d'une même ligne qui se suivent, On considère que l'on change de mot.

On en profite (comme pour la grille) pour donner une position de lettre dans un mot (colonne) et l'id du mot qui correspond à la lettre (ligne).

4.1.4 Suppressions des énumérations et des mots courts

On compare l'écart d'espacement entre la première et la deuxième composante avec la moyenne des écarts d'espacement entre toutes les lettres du même mot.

Si la différence dépasse une tolérance en x , alors il y a un symbole devant ce mot (tiret, numéro de mot, cercle ou tout autre symbole qui énumère les mots). Les positions des lettres peuvent alors être décalées par les symboles en position 0.

Si il existe un mot qui compte moins de 2 lettres, on supprime toutes ses lettres. Il peut donc avoir aussi un décalage entre les ids des mots.

On recalcule alors la véritable position des lettres dans chaque mot ainsi que l'id du mot. Les lettres dans un mot sont triées par leur centre en x .

Les ids des mots redeviennent un nombre entre 0 et $nombre_de_mots - 1$.

4.1.5 Lettres collées

Dans les images d'origine, il arrive parfois que plusieurs lettres soient collées ou connectées visuellement (surtout dans le dernier niveau).

Exemples typiques : LA, RA, MA, HA, YT, TT, PT, UN, LAA.

Séparer les lettres à cette étape ne permet pas de savoir combien de lettres il y a dans une composante connexe. Cependant, si le réseau de neurones ne parvient pas à trouver une lettre, c'est quelle doit être séparée en $n + 1$ morceaux.

Afin de transmettre à l'IA un caractère isolé, il est indispensable de découper correctement ces groupes.

Le but est de trouver les positions verticales où découper une image contenant plusieurs lettres jointes.

L'algorithme repose sur un histogramme vertical, où chaque colonne compte le nombre de pixels noirs. Les colonnes présentant peu de pixels noirs correspondent généralement à des vallées, c'est à dire des zones où les lettres pourraient être séparées.

Au lieu d'utiliser un seuil fixe, l'algorithme cherche la valeur minimale de l'histogramme puis construit un seuil adaptatif en fonction des vallées. On évite les vallées trop nombreuses ou trop petites. On ajuste le seuil si les vallées trouvées semblent incohérentes.

La fonction repère les intervalles où l'histogramme est inférieur au seuil. Chaque intervalle donne une vallée candidate, dont on extrait la position médiane :

$$cands[ccount + +] = (vstart + vend)/2;$$

Les bords de l'image sont explicitement exclus pour éviter des découpes invalides. Pour un découpage en nb_cut coupes, les coupes idéales seraient uniformément réparties par $step = width/(nb_cut + 1)$.

Pour chaque position idéale, on cherche la vallée la plus proche. La position finale choisie est, pour une meilleure stabilité, la moyenne entre le centre de la vallée et la position idéale.

4.2 Outil d'analyse de grilles d'images

Le programme "Create_Grid_txt.c" est un outil d'analyse et de visualisation conçu pour traiter des collections de fichiers image. Son objectif principal est de générer une représentation visuelle sous forme de grille textuelle montrant la présence et la disposition spatiale de fragments d'images extraits d'une image source plus large.

4.2.1 Architecture, fonctionnement et principe de base

Le programme analyse récursivement un répertoire et ses sous-dossiers à la recherche de fichiers BMP dont le nom encode des informations de position. Ces fichiers suivent une convention de nommage très précise qui contient les coordonnées (ligne et colonne) de chaque fragment dans une grille globale.

Pour chaque fichier trouvé, le programme extrait ces coordonnées, puis construit une grille de caractères où chaque position contenant un fichier est marquée par un 'X', tandis que les positions vides restent des espaces. Cette grille est ensuite sauvegardée dans un fichier texte, offrant ainsi une vue d'ensemble immédiate de la couverture spatiale de la collection de fichiers.

4.2.2 Convention de nommage des fichiers

Le programme s'attend à trouver des fichiers dont le nom suit ce format strict :
grid_letter_level_1_image_1.png_bw.bmp_[id]_[col]x[row]_(dimensions).bmp

Par exemple :

grid_letter_level_1_image_1.png_bw.bmp_100_15x5_(718x203_10x17).bmp

Dans cet exemple :

grid_letter_level_1_image_1.png_bw.bmp : nom de l'image source

100 : identifiant unique du fragment

15x5 : position dans la grille (colonne 15, ligne 5)

(718x203_10x17) : coordonnées des pixels dans l'image

.bmp : extension du fichier

4.2.3 Fonction d'extraction de position

La fonction `extraire_position()` constitue le cœur de l'analyse des noms de fichiers. Son algorithme procède en plusieurs étapes :

- Localisation du marqueur : elle cherche d'abord la chaîne `.png_bw.bmp_` qui sert de point de repère dans le nom du fichier.
 - Navigation dans le nom : elle avance dans la chaîne de caractères en sautant l'identifiant numérique jusqu'au prochain underscore.
 - Extraction des coordonnées : elle utilise `sscanf()` pour parser le format `[col]x[row]_` et extraire les deux entiers.
 - Validation : elle retourne 1 en cas de succès, 0 si le format ne correspond pas.
- Cette approche robuste permet de gérer des noms de fichiers complexes tout en se concentrant uniquement sur les informations pertinentes.

4.2.4 Fonction de comparaison pour le tri

La fonction `comparer_positions()` implémente la logique de tri nécessaire pour `qsort()`. Elle établit un ordre lexicographique sur les positions :

- Priorité principale : tri par numéro de ligne (`row`)
- Priorité secondaire : tri par numéro de colonne (`col`)

Ce tri est crucial car il permet de placer les positions dans un ordre logique, facilitant la construction cohérente de la grille finale.

4.2.5 Parcours récursif des dossiers

La fonction `parcourir_dossiers()` implémente une exploration des fichiers. Son fonctionnement :

- Ouverture du répertoire : utilisation de `opendir()` pour accéder au contenu
- Itération sur les entrées : lecture successive de chaque élément via `readdir()`
- Filtrage : exclusion des entrées spéciales `"."` et `".."`
- Décision récursive :
 - Si l'entrée est un dossier (`DT_DIR`), appel récursif sur ce sous-répertoire
 - Si l'entrée est un fichier BMP, tentative d'extraction de la position
- Accumulation : stockage des positions valides dans un tableau Cette approche garantit que tous les fichiers BMP sont trouvés.

4.2.6 Construction et écriture de la grille

La fonction `creer_grille()` orchestre l'ensemble du processus :

- Collecte des données : appel de la fonction de parcours pour remplir le tableau de positions
- Tri : organisation des positions selon l'ordre défini
- Calcul des dimensions : détermination des bornes minimales et maximales pour row et col
- Normalisation : calcul des dimensions de la grille en soustrayant les minimums
- Allocation mémoire : création dynamique d'une matrice de caractères
- Initialisation : remplissage de la grille avec des espaces
- Marquage : placement des 'X' aux positions normalisées où des fichiers existent
- Écriture : sauvegarde ligne par ligne dans le fichier de sortie
- Libération : nettoyage de la mémoire allouée

4.2.7 Quelques statistiques du programme

Le programme ne se contente pas de générer la grille, il fournit également des informations statistiques précieuses :

- Nombre total de fichiers trouvés
- Nombre de lignes (minimum et maximum)
- Nombre de colonnes (minimum et maximum)
- Dimensions calculées de la grille
- Comparaison entre le nombre de positions attendues (produit des dimensions) et le nombre de fichiers réellement trouvés

Ces statistiques permettent une évaluation quantitative rapide de la complétude du dataset. Un écart important entre le nombre de positions attendues et le nombre de fichiers trouvés signale immédiatement un problème nécessitant investigation.

4.2.8 Les problèmes rencontrés lors de l'implémentation

Durant l'implémentation de ce programme, certains problèmes ont été rencontrés. On peut citer en premier lieu, l'une des erreurs majeures comme la mauvaise création de la grille. Cela provenait du fait que les lignes et les colonnes étaient inversés dans le code et par exemple au lieu d'avoir une grille 16x23, on avait une grille 23x16.

Un autre problème rencontré est une mauvaise création de la grille. Pour contexte, on récupère le dossier des images découpées mais si une image est mal découpée car les pixels de deux lettres sont trop proches alors l'image résultat ne créera qu'un X dans le programme au lieu de deux.

Ce problème se résout néanmoins grâce à un meilleur découpage de l'image comme par exemple l'image "LAA" qui contient trois lettres. Cette solution est notamment nécessaire pour l'IA qui va donc avoir de meilleurs entraînements. Cette solution va donc être appliquée à notre programme quand l'IA se chargera d'utiliser la grille.

4.2.9 Résumé du programme

Le programme `Create_Grid_txt.c` illustre comment un outil simple peut résoudre efficacement un problème spécifique de gestion et de vérification de collections de fichiers. Sa force réside dans sa capacité à transformer un ensemble potentiellement complexe et volumineux de fichiers en une représentation visuelle immédiatement compréhensible.

Dans notre cas, il permet surtout de construire une grille pour les lettres de la grille et les lettres des mots afin de faciliter la lecture via le format fichier texte.

Cependant il faut bien prendre en compte que le programme dépend entièrement de la convention de nommage spécifique. Tout écart dans le format rend le fichier invisible pour l'analyse. Cela est possible uniquement grâce au nommage spécifique au moment de la découpe des lettres de la grille et des mots.

Voici quatre captures d'écran. La première montre la détection des fichiers pour les lettres de la grille, ces statistiques et la création du fichier. La deuxième fait la même chose mais avec les mots de l'image à chercher dans la grille. La troisième correspond à la grille créée et la quatrième est l'exemple de mots de l'image `_1_level_1` à chercher.

```
Trouvé: grid_letter_level_1_image_1.png_bw.bmp_178_8x10_(476x376_9x17).bmp -> row=8, col=10
Nombre de fichiers trouvés: 289
Statistiques:
  Row: min=0, max=16
  Col: min=0, max=16
Dimensions de la grille: 17 lignes x 17 colonnes
Positions attendues: 289 (fichiers trouvés: 289)
Grille créée dans grille.txt
```

```
Trouvé: word_letter_level_1_image_1.png_bw.bmp_18_2x3_(57x219_12x20).bmp -> row=2, col=3
Nombre de fichiers trouvés: 51
Statistiques:
  Row: min=0, max=6
  Col: min=0, max=8
Dimensions de la grille: 9 lignes x 7 colonnes
Positions attendues: 63 (fichiers trouvés: 51)
Grille créée dans grille.txt
```

```
XXXXXXX
XXXXX
XXXX
XXXXXXX
XXXXXXX
XXXX
XXXXXXX
XXXXXX
XXXX
```

IMAGINE
RELAX
COOL
RESTING
BREATHE
EASY
TENSION
STRESS
CALM

Sur ces deux dernières captures d'écrans, on reconnaît bien le nombre de lettres par mots et de manière évidente, on voit que la première ligne de X dans la grille correspond au mot "IMAGINE" à chercher et donc de même pour les mots suivants.

4.3 Affichage de la résolution de la grille

4.3.1 Annotation de l'image résolue

L'ensemble de la chaîne est automatisée, depuis l'extraction des lettres jusqu'au rendu final annoté.

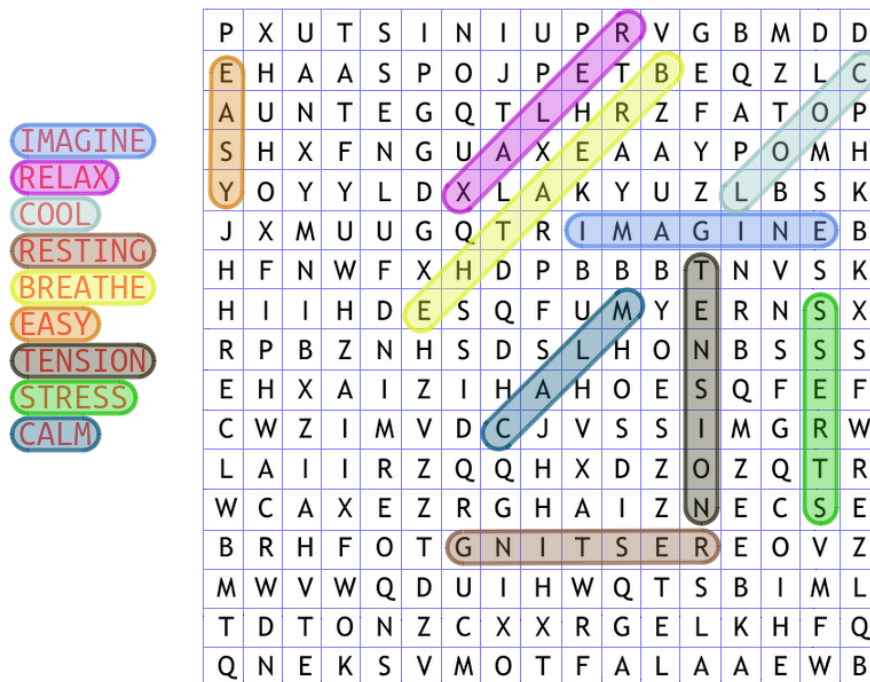
Le but est de rassembler plusieurs modules précédemment expliqués, pour générer une image annotée où chaque mot trouvé est entouré par une forme professionnelle et esthétique, proche des applications modernes de jeux de lettres.

On récupère d'abord les résultats du solveur pour chaque mot trouvé, qui sont transférés dans un fichier texte.

Les positions sur l'image de base sont obtenus grâce à un parseur.

On crée des couleurs aléatoires et reproductibles pour chaque mot en fonction de son id. On colorise un mot à trouver de la même couleur que celui du mot qui lui correspond dans la grille. En cas de mot non trouvé par le solver, une ligne sombre simple est tracée sur le mot de la liste à trouver en diagonale pour le barrer.

On exporte finalement le fichier final automatiquement.



Grille résolue

4.3.2 Surlignage et anticrénelage

On forme des rectangles qui peuvent être inclinés avec des coins arrondis correctement orientés. Toutes ces opérations sont calculées dans un repère géométrique local, basé sur le vecteur directeur du mot (dx, dy) , un vecteur normal (nx, ny) et les quatre coins de la forme.

On remplit avec une super-ellipse de même couleur et semi-transparente pour faire ressortir le mot. On peut modifier la forme de l'arrondi :

forme carrée, parenthèse, arrondi, capsule, losange ou très pointu.

Seuls les coins sont transformés, le centre reste un rectangle.

Cela permet d'obtenir une forme stylisée sans déformer la zone principale.

Anticrénelage (lissage des obliques ou lissage de police) logiciel (numérique) :

L'algorithme permet de déterminer graduellement la couleur des pixels aux frontières entre deux zones de teinte différentes. Le choix de la couleur d'un pixel se fait donc en fonction de sa proximité avec les deux zones colorées. Après l'application de ce processus, la démarcation entre les deux zones apparaîtra un peu plus floue, mais n'aura plus d'allure de créneau.

On utilise cette technique pour les arcs et les lignes épaisses.

Les bords paraissent alors plus lisses et adoucis.

4.4 Rotation automatique de l'image

L'un des défis techniques majeurs que j'ai relevé dans ce projet a été la mise en place d'un système de rotation automatique capable de redresser des images inclinées. Dans le contexte de notre OCR, il arrive fréquemment que les photos de grilles de mots cachés soient prises avec un léger angle, ce qui perturbe considérablement la détection des lettres et la reconnaissance par le réseau neuronal. Il était donc essentiel de développer une solution robuste pour corriger automatiquement ces défauts d'alignement.

4.4.1 Le principe de détection de l'angle optimal

L'algorithme que j'ai développé repose sur une méthode d'analyse par projection horizontale. L'idée fondamentale est simple : une image correctement alignée présente des lignes de texte parfaitement horizontales. Si l'on projette tous les pixels noirs sur l'axe vertical (c'est-à-dire si l'on compte combien de pixels noirs se trouvent sur chaque ligne), on obtient un histogramme avec des pics marqués correspondant aux lignes de texte.

Plus ces pics sont prononcés et réguliers, plus l'image est droite. À l'inverse, si l'image est inclinée, les pixels noirs se répartissent de manière plus uniforme sur l'axe vertical, car les lignes de texte sont "étalées" sur plusieurs lignes de pixels. En calculant la variance de cet histogramme de projection, on obtient un score d'alignement : une variance élevée indique une bonne séparation entre les lignes (donc une image droite), tandis qu'une variance faible suggère que l'image est inclinée.

Pour trouver l'angle de rotation optimal, l'algorithme teste systématiquement différents angles de rotation dans une plage définie (de -45° à $+45^\circ$). Pour chaque angle testé, l'image est temporairement pivotée, puis le score d'alignement est calculé. L'angle qui produit le score le plus élevé est considéré comme l'angle de redressement idéal.

4.4.2 Optimisation de la recherche d'angle

Tester tous les angles possibles avec une précision fine serait extrêmement coûteux en temps de calcul. Pour résoudre ce problème, j'ai implémenté une stratégie de recherche en deux phases : une phase de balayage grossier suivie d'une phase d'affinage.

Dans la première phase, l'algorithme teste des angles espacés de 3 degrés (par exemple -45° , -42° , -39° , etc.). Cette exploration rapide permet d'identifier grossièrement la zone où se situe l'angle optimal. Une fois cette zone repérée, l'algorithme entre dans la seconde phase : il effectue une recherche fine dans une fenêtre de $\pm 3^\circ$ autour de l'angle grossier détecté, avec un pas de 1 degré. Cette approche hybride permet d'obtenir une précision d'un degré tout en réduisant considérablement le nombre de calculs nécessaires.

De plus, pour accélérer encore le traitement, l'analyse est effectuée sur une version redimensionnée de l'image (limitée à 600 pixels de largeur). Les calculs de score sont donc très rapides, et seule la rotation finale est appliquée à l'image en pleine résolution, garantissant ainsi une qualité optimale du résultat.

4.4.3 Résolution de l'ambiguïté de signe

Un problème subtil se pose lors de la détection d'angle : l'algorithme peut détecter un angle de, disons, $+15^\circ$, mais il n'est pas toujours évident de savoir si l'image doit être tournée de $+15^\circ$ ou de -15° pour être redressée. En effet, les deux rotations produisent des images visuellement différentes, mais l'une redresse l'image tandis que l'autre l'incline davantage.

Pour lever cette ambiguïté, j'ai ajouté une phase de vérification finale : une fois l'angle optimal détecté (par exemple $+15^\circ$), l'algorithme teste à la fois $+15^\circ$ et -15° , calcule le score d'alignement pour chacun, et choisit l'angle qui produit le meilleur score. Cette vérification garantit que l'image est toujours redressée dans la bonne direction, quelle que soit son orientation initiale.

4.4.4 Intégration avec ImageMagick

Pour implémenter cet algorithme, j'ai utilisé la bibliothèque ImageMagick (MagickWand), qui offre des fonctionnalités puissantes de manipulation d'images. Cette bibliothèque permet de charger des images en mémoire, d'effectuer des rotations, de récupérer les valeurs de pixels et d'exporter le résultat, le tout avec d'excellentes performances.

Le programme `rotate_bmp_im.c` que j'ai développé prend en entrée une image BMP, effectue toute l'analyse décrite précédemment, et sauvegarde l'image redressée sous le nom `final_rotated.bmp`. Ce programme est ensuite appelé par l'interface graphique via un appel système, créant ainsi un pont entre l'interface utilisateur et le module de traitement d'image.

Un soin particulier a été apporté à la gestion des erreurs et à la qualité de l'image finale. Après rotation, l'image peut parfois présenter des bandes blanches sur les bords (zones ajoutées pour accommoder la nouvelle géométrie). J'ai donc implémenté un système de recadrage intelligent qui supprime ces bandes excessives tout en préservant l'intégralité du contenu utile de l'image.

4.4.5 Défis techniques et solutions apportées

L'un des premiers défis que j'ai rencontrés a été la gestion correcte de la mémoire pour les pixbufs GTK. Un pixbuf est une représentation en mémoire d'une image, et chaque fois qu'une nouvelle image est chargée, il faut s'assurer de libérer l'ancien pixbuf pour éviter les fuites mémoire.

J'ai implémenté un système rigoureux de gestion de la mémoire : avant de charger une nouvelle image, l'interface vérifie si un pixbuf original existe déjà (`app->original_pixbuf`). Si c'est le cas, il est libéré avec `g_object_unref()` avant d'être remplacé par le nouveau. Cette pratique, bien qu'apparemment simple, est cruciale pour assurer la stabilité de l'application lors d'une utilisation prolongée.

Un autre défi important a été de coordonner l'exécution des programmes externes. Lorsque l'interface appelle un programme via `system()`, elle doit attendre que ce programme termine son exécution avant de tenter de charger le fichier résultat. Heureusement, `system()` est un appel bloquant : l'interface attend automatiquement la fin du programme avant de continuer.

Cependant, j'ai dû gérer les cas où un programme externe échoue ou produit une erreur. Pour cela, je vérifie systématiquement le code de retour de `system()` : un code 0 indique un succès, tandis qu'un code différent signale une erreur. En cas d'échec, l'interface affiche un message explicite et n'essaie pas de charger un fichier qui n'existe probablement pas.

Au fil du développement, j'ai dû adapter l'interface à plusieurs reprises pour l'aligner avec l'évolution du projet. Par exemple, j'avais initialement implémenté un `spinbutton` permettant à l'utilisateur de faire pivoter manuellement l'image. Cependant, après la mise en place de la rotation automatique, cette fonctionnalité est devenue redondante et a été supprimée pour simplifier l'interface.

J'ai également ajouté des boutons dans la barre d'en-tête (`header bar`) pour faciliter l'accès aux fonctions les plus courantes (importation et sauvegarde). Cette décision de design améliore l'ergonomie en offrant plusieurs points d'accès aux fonctionnalités principales, permettant à l'utilisateur de travailler plus efficacement.

4.5 Intégration complète de la chaîne de traitement

L'un des aspects les plus gratifiants de mon travail sur l'interface a été de rassembler tous les modules du projet en un système cohérent et automatisé. Chaque composant développé par les membres de l'équipe devrait pouvoir être appelé de manière fluide depuis l'interface graphique.

4.5.1 Le flux de traitement automatisé

J'ai conçu l'interface de manière à ce que l'utilisateur puisse suivre visuellement la progression du traitement. Lorsqu'il clique sur « Rotation automatique », plusieurs opérations s'enchaînent automatiquement :

D'abord, l'image actuellement affichée est sauvegardée temporairement (`temp_input.bmp`). Ensuite, le programme de rotation automatique est appelé et effectue son analyse. Une fois la rotation terminée, l'image redressée (`final_rotated.bmp`) est chargée et remplace l'image précédente dans l'interface. Mais ce n'est pas tout : l'image redressée est également sauvegardée sous un nom standardisé (`gui_input_rotated.png`) pour les traitements suivants.

Immédiatement après, l'interface lance automatiquement le programme `preprocess` pour convertir l'image en noir et blanc, produisant `gui_input_rotated.png_bw.bmp`. Sans intervention de l'utilisateur, le programme `get_letters` est ensuite appelé pour détecter et extraire les lettres de la grille. Toute cette chaîne de traitement s'exécute en quelques secondes, et l'utilisateur peut suivre la progression grâce aux messages affichés dans la console.

Cette automatisation représente un gain de temps considérable. L'utilisateur n'a pas à se soucier des détails techniques : il charge une image, clique sur un bouton, et le système prend en charge tout le reste. C'est exactement ce que nous voulions accomplir avec ce projet : rendre l'OCR accessible et simple d'utilisation.

4.5.2 Gestion des chemins et des dépendances

Un défi technique important a été la gestion des chemins de fichiers relatifs et absolus. Chaque module du projet a été développé indépendamment, avec ses propres conventions de nommage et d'organisation des fichiers. Par exemple, `preprocess` s'attend à trouver ses images dans `image_modifier/tests_images/`, tandis que le programme de rotation fonctionne dans `interface/automatic_rotation/`.

J'ai dû m'assurer que tous ces chemins étaient cohérents et que les fichiers intermédiaires étaient créés aux bons endroits. Pour cela, j'ai utilisé des commandes `cd` dans les appels système, permettant de changer le répertoire de travail avant d'exécuter chaque programme externe. Cette approche, bien que simple, s'est révélée très efficace et robuste.

De plus, j'ai mis en place un système de vérification des erreurs à chaque étape. Si un programme externe échoue (par exemple si un fichier est introuvable), l'interface capture le code de retour et affiche un message d'erreur explicite dans la console, permettant à l'utilisateur de comprendre ce qui s'est passé et de corriger le problème.

4.6 Système neuronal

4.6.1 Normalisation des Images d'Entrée

Le système d'apprentissage des neurones débute par une étape fondamentale : la normalisation des images. En effet, les images fournies en tant que données d'entraînement ne possèdent pas toutes les mêmes dimensions. Certaines lettres peuvent être très grandes tandis que d'autres sont minuscules. Pour pallier ce problème de variabilité de taille, nous avons décidé de convertir l'ensemble des images dans un format standardisé de 45×45 pixels.

Cette normalisation présente plusieurs avantages. D'abord, elle garantit que chaque image, qu'elle contienne une lettre de petite ou grande taille, peut être traitée de manière uniforme par le réseau neuronal. Ensuite, elle permet de maintenir une cohérence dans les données d'entrée, ce qui facilite l'apprentissage et améliore la stabilité du modèle.

Durant cette conversion, l'image originale est centrée dans la matrice de 45×45 pixels. Les espaces vides autour de l'image sont complétés par des pixels blancs, créant ainsi un cadre régulier. Cette approche préserve les caractéristiques visuelles de la lettre tout en respectant le format requis par le réseau de neurones

4.6.2 Architecture du Réseau Neuronal

Le réseau neuronal mis en place pour cette reconnaissance de lettres adopte une architecture en trois couches : une couche d'entrée, une couche cachée et une couche de sortie.

Couche d'Entrée: La couche d'entrée reçoit les données provenant de l'image normalisée de 45×45 pixels. Cela représente un total de 2 025 neurones d'entrée ($45 \times 45 = 2\,025$), chacun correspondant à un pixel de l'image. Chaque pixel est converti en valeur numérique : les pixels sombres (représentant la lettre) sont codifiés comme 1.0, tandis que les pixels clairs (l'arrière-plan blanc) sont codifiés comme 0.0.

Couche Cachée: Entre la couche d'entrée et la couche de sortie se trouve la couche cachée, composée de 150 neurones. Ces neurones cachés jouent un rôle crucial : ils permettent au réseau d'apprendre des caractéristiques complexes et abstraites des lettres. Plutôt que de simplement analyser les pixels individuels, la couche cachée combine ces informations pour identifier des motifs, des formes et des structures communes aux différentes lettres. Les 150 neurones cachés représentent un équilibre judicieux entre la capacité d'apprentissage et l'efficacité computationnelle. Un nombre trop faible de neurones cachés limiterait la capacité du réseau à apprendre des distinctions fines entre les lettres, tandis qu'un nombre excessif risquerait de conduire au surapprentissage.

Couche de Sortie: La couche de sortie comporte 26 neurones, correspondant aux 26 lettres de l'alphabet (de A à Z). Chaque neurone de sortie produit une valeur entre 0 et 1, représentant la probabilité que l'image entrante corresponde à la lettre associée. Lors de la prédiction, la lettre reconnue est celle dont le neurone de sortie affiche la valeur la plus élevée.

4.6.3 Fonctionnement Général du Système

Le système fonctionne selon deux phases distinctes : la phase d'entraînement et la phase de prédiction.

Phase d'Entraînement: Lors de l'entraînement, pour chaque image de la base de données d'apprentissage, le réseau effectue une propagation avant (forward pass) qui calcule les sorties prédites. L'erreur entre ces prédictions et les résultats attendus (one-hot encoding où la lettre correcte vaut 1 et les autres valent 0) est ensuite calculée. Cette erreur est rétropropagée à travers le réseau grâce à l'algorithme de backpropagation, permettant d'ajuster les poids et les biais de chaque neurone. Ce processus se répète sur plusieurs générations (epochs) jusqu'à ce que le réseau converge vers une bonne précision de reconnaissance.

Phase de Prédiction: Une fois le réseau entraîné, il peut être utilisé pour reconnaître de nouvelles images de lettres. L'image est d'abord normalisée au format $45 \times$

45, puis propagée à travers le réseau. Le neurone de sortie présentant la valeur maximale indique la lettre prédite.

4.6.4 Mécanisme de Récompense et de Punition

Un aspect fondamental du système d'apprentissage est le mécanisme de récompense et de punition qui guide l'ajustement des poids du réseau. Contrairement à une approche classique, ce mécanisme n'utilise pas de système de bonus/pénalités externes explicites, mais plutôt un système mathématique basé sur l'erreur et la rétropropagation.

Calcul de l'Erreur (Punition):

L'apprentissage commence par le calcul de l'erreur pour chaque neurone de sortie. Cette erreur mesure à quel point la prédiction du réseau s'éloigne de la réalité. Pour chaque neurone de sortie k , l'erreur est définie par :

$$\text{Erreur}_k = \text{Valeur_Cible}_k - \text{Prédiction}_k$$

- Valeur_Cible_k est la valeur attendue (1 si c'est la bonne lettre, 0 sinon)

- Prédiction_k est la sortie du neurone (valeur entre 0 et 1)

Calcul du Gradient de Sortie (Delta):

L'erreur brute est multipliée par la dérivée de la fonction d'activation (sigmoid) pour créer le gradient de sortie, appelé delta

$$\Delta_k = \text{Erreur}_k \times \sigma'(\text{Sortie}_k)$$

$-\sigma'(Sortie_k)$ est la dérivée de la sigmoïde : $\sigma'(x) = x \times (1 - x)$

Ce delta représente l'amplitude et la direction de l'ajustement nécessaire. Une grande erreur produit un grand delta, ce qui entraînera des ajustements importants des poids.

Rétropropagation vers la Couche Cachée:

L'erreur est rétropropagée vers les neurones cachés. Chaque neurone caché j reçoit une contribution d'erreur de tous les neurones de sortie :

$$\text{Erreur_Cachée}_j = \sum_k \Delta_k \times W_{j,k}$$

Puis le delta pour la couche cachée est calculé de façon similaire :

$$\Delta_j = \text{Erreur_Cachée}_j \times \sigma'(\text{Sortie_Cachée}_j)$$

Interprétation : Récompense ou Puniton:

Récompense (ajustement positif) : Lorsque le réseau se trompe et produit une erreur, le delta est grand. Si un poids contribue positivement à la bonne direction, il sera augmenté. C'est la récompense : les connexions qui aident à corriger l'erreur sont renforcées.

Puniton (ajustement négatif) : Inversement, si un poids contribue à l'erreur, il sera diminué. Les connexions qui ont contribué à la mauvaise prédiction sont affaiblies. C'est la puniton.

Magnitude de l'ajustement : Plus l'erreur est grande, plus l'ajustement des poids est important. Un taux d'apprentissage faible (0.005) assure des ajustements graduels, évitant des changements trop brusques qui pourraient déstabiliser l'apprentissage.

Avec ce delta important, les poids qui ont contribué à cette sortie faible seront augmentés (récompense pour les bonnes connexions) ou diminués (punition pour les mauvaises) selon leur contribution respective.

4.6.5 Sauvegarde et Stockage des Poids du Réseau Neuronale

Après l'entraînement, l'étape cruciale de sauvegarde des poids permet de conserver le modèle pour une utilisation ultérieure. Le système enregistre quatre ensembles de paramètres essentiels dans un fichier texte structuré.

Structure du Fichier de Sauvegarde

Le fichier de sauvegarde est organisé en quatre sections clairement délimitées par des en-têtes :

Section 1 : WEIGHTS_IH (Poids entrée-vers-caché) Cette matrice de $2\,025 \times 150$ valeurs représente les 303 750 connexions entre chaque neurone d'entrée et chaque neurone de la couche cachée. Elle est écrite ligne par ligne, avec 150 valeurs par ligne.

Section 2 : BIASES_H (Biais cachés) Un vecteur de 150 valeurs, une pour chaque neurone caché. Ces biais ajustent le seuil d'activation de chaque neurone caché indépendamment.

Section 3 : WEIGHTS_HO (Poids caché-vers-sortie) Une matrice de 150×26 valeurs représentant les 3 900 connexions entre les neurones cachés et les 26 neurones de sortie (un par lettre). Elle est écrite ligne par ligne, avec 26 valeurs par ligne.

Section 4 : BIASES_O (Biais de sortie) Un vecteur de 26 valeurs, un pour chaque neurone de sortie correspondant à une lettre de l'alphabet.

Détails de l'Enregistrement

Précision des valeurs : Chaque poids et biais est enregistré avec 10 décimales. Cette précision est essentielle car même de petites variations peuvent affecter les prédictions du réseau. La notation scientifique et les chiffres décimaux permettent de capturer des valeurs très petites (proches de zéro) ou très grandes.

Organisation en lignes : Les poids sont écrits ligne par ligne pour une lisibilité et une maintenance faciles. Par exemple, chaque ligne de **WEIGHTS_IH** contient 150 valeurs correspondant aux 150 connexions d'un neurone d'entrée vers tous les neurones cachés.

En-têtes descriptifs : Chaque section commence par un en-tête clair (**— WEIGHTS_IH** —, etc.) qui permet de valider l'intégrité du fichier lors du chargement ultérieur.

Volume de Données Sauvegardées

Le nombre total de paramètres sauvegardés est :

Poids IH : $2\,025 \times 150 = 303\,750$ valeurs

Biais H : 150 valeurs

Poids HO : $150 \times 26 = 3\,900$ valeurs

Biais O : 26 valeurs

Total : 307 826 paramètres

Chaque paramètre occupe environ 12 caractères (10 chiffres et espacement), ce qui représente un fichier d'environ 3,7 mégabytes. Bien que volumineux en format texte, cette approche offre l'avantage de pouvoir inspecter et modifier les poids manuellement si nécessaire.

Intégrité et Validation

Après la sauvegarde, le fichier peut être validé par :

- 1-Vérification de la présence de tous les en-têtes attendus
- 2-Comptage du nombre de valeurs dans chaque section
- 3-Vérification que toutes les valeurs sont des nombres valides

Ces paramètres sauvegardés représentent tout le « savoir » que le réseau a acquis lors de l'entraînement. Ils peuvent ensuite être réutilisés pour reconnaître de nouvelles images sans avoir besoin de réentraîner le modèle.